A Sink-Guided Agentic Scaffold for Defensive Vulnerability Discovery: An Outside-In Study Inspired by Project Glasswing

Mythos Research Edition – a reproducible scaffold around a general-purpose frontier model, with cost telemetry from real-world deployment

Keyvan Hardani*

April 2026
Version 2.1 (Research Edition v4 scaffold, arXiv-prep)

Abstract

Anthropic's April 2026 *Mythos Preview* announcement (Project Glasswing) reports large gains in autonomous vulnerability discovery, attributed to a specialised model checkpoint paired with a comparatively simple agentic scaffold [5]. We *reproduce publicly described scaffold elements*, not performance claims: we do not have access to the specialised checkpoint and we do not measure ourselves against the published Glasswing numbers. Using a general-purpose frontier model (Claude Opus 4.7, Anthropic 6), we implement an eight-phase pipeline—language detection, sink slicing, file ranking, build-sandbox preparation, live-tool agentic hunting, adversarial self-challenge, sceptical validation, and aggregation, with a held-private live-execution stage referenced but not shipped—that targets the publicly described workflow elements of Glasswing. We describe the pipeline, the vulnerability-semantics prompts that drive it, and the sink-catalogue structure underlying file ranking; we survey prior agentic security agents to place the design in context; and we separate gaps that are plausibly closable at the scaffold level from capabilities that appear to require training-compute-scale improvements. We report aggregate cost telemetry and a small qualitative deployment record from real-world coordinated-disclosure work. We deliberately do not present a controlled head-to-head benchmark, and discuss why building an agentic-pipeline-appropriate benchmark is itself an open research problem.

Keywords—large language models; vulnerability discovery; agentic scaffolding; prompt engineering; sink analysis; coordinated disclosure; Claude; Glasswing.

*Independent Applied AI Research. Contact: hello@keyvan.ai. ORCID 0009-0000-6003-8826. Source code, prompts, sink catalogues, and LaTeX source of this report: github.com/Keyvanhardani/mythos-research. Archived at Zenodo, concept DOI 10.5281/zenodo.19727857 (version DOI for v1.0.2: 10.5281/zenodo.19727858).

*Scope and honesty statement.*This report is an *outside-in* replication of the publicly described scaffold. It does not reproduce the Mythos Preview checkpoint itself. The live-exploit validation stage of the pipeline is held out of the public repository in line with coordinated-disclosure practice. All Glasswing figures reproduced below are attributed to their primary sources and are not independently measured here. Specific advisory identifiers, project names, and per-finding details from the operator's coordinated-disclosure work are deliberately omitted from this report; the maintainer's public profile carries that record as embargoes lift.

Contents

1 Introduction

1.1 Motivation

Anthropic's Mythos Preview announcement reports that a specialised model checkpoint paired with a simple agentic scaffold materially advances autonomous vulnerability discovery, including a full autonomous exploit of CVE-2026-4747 in FreeBSD NFS at a per-run cost below $50 [5, 15]. Two contributions are bundled in that announcement: the *model* and the *scaffold*. The announcement attributes most of the capability gain to the model, but the scaffold is described in sufficient public detail to invite replication.

We examine the scaffold in isolation and ask whether its publicly described workflow elements can be reproduced with a general-purpose frontier model (Claude Opus 4.7) on commodity

hardware, at what cost, and with what capability boundary. The question matters because the scaffold design is what practitioners can reuse today, while access to the specialised checkpoint is limited.

1.2 Research questions

We structure the report around three questions:

RQ1.
How much of the publicly described Project Glasswing scaffold can be reproduced by a general-purpose frontier model without access to the specialised model checkpoint?

RQ2.
Which vulnerability-finding capabilities does the primary source attribute to the model checkpoint versus to scaffold-level components, and where on that boundary does a sink-guided, sceptically-validated pipeline running on a general-purpose model land?

RQ3.
What per-run cost profile does such a pipeline achieve against self-scannable open-source targets, and what is the cost–quality trade-off relative to the Glasswing cost figures reported in the primary source?

RQ1 is answered by the design of the replicated scaffold in §4. RQ2 is answered by the capability-boundary analysis in §7, drawing on the primary source's reported figures, the public literature on agentic security agents, and the operator's qualitative observations from real-world deployment of the pipeline. RQ3 is addressed by the cost telemetry in §6.1, which reports aggregate per-run cost from real coordinated-disclosure audits.

1.3 Contributions

We contribute:

C1. An open-source, sink-guided, eight-phase agentic pipeline for defensive vulnerability discovery, built around a general-purpose frontier model and released under Apache-2.0 (artefact). The current default (`mythos-v4.sh`) adds a build-sandbox preparation phase, a live-tool agentic hunter, an adversarial self-challenge stage, and cross-session false-positive memory over the v3.1 baseline.

C2. Language-specific sink catalogues for C/C++, Python, PHP, and JavaScript/TypeScript, with explicit `SAFE_*`-variant demotion to reduce false positives (artefact).

C3. A vulnerability-semantics-guided hunter prompt paired with a sceptical validator prompt and an adversarial self-challenge prompt, designed to produce JSON-structured findings with explicit source-to-sink traceability (artefact).

C4. A capability-boundary analysis separating scaffold-closable gaps from weight-level ceilings, grounded in both the primary source's reported figures and the prior literature on agentic security agents (analysis).

C5. Aggregate cost telemetry from real-world deployment of the pipeline in coordinated-disclosure audits, alongside a discussion of why a controlled head-to-head benchmark for agentic pipelines remains an open problem (observations and methodological discussion).

1.4 Outline

§2 summarises the publicly described Mythos Preview scaffold and fixes terminology. §3 positions the work against prior agentic security agents. §4 describes the replicated pipeline. §5 covers implementation and reproducibility. §6 reports empirical deployment observations and cost telemetry. §7 presents the capability-boundary analysis. §8–§10 close with ethics, limitations, and a conclusion.

2 Background and Problem Setting

2.1 Agentic vulnerability discovery

Agentic vulnerability discovery describes a pattern in which a large language model orchestrates a fixed set of deterministic tools—typically a code browser, a debugger with memory-safety instrumentation, and a constrained shell or Python sandbox—to explore a target codebase, form hypotheses, test them, and emit structured findings. Prior instances include Project Naptime and its successor Big Sleep [8, 9], the semantics-scoped patching framework APPATCH [16], the code-property-graph driven LLMxCPG [11], and CVE-Genie's developer–critic loop [7]. The common design principle across these systems is that the model orchestrates deterministic tools that provide ground-truth feedback, rather than acting as the verifier itself.

2.2 The Mythos Preview scaffold as described

The public Mythos Preview announcement describes a scaffold that is deliberately simple [5]:

- an isolated container is launched with the target project's source code;
- the model is prompted, with a trivial discovery prompt, to find a security vulnerability;
- the model autonomously reads code, forms hypotheses, executes the project under debuggers and custom debug logic, and produces a structured bug report with a proof-of-concept exploit;
- files are ranked 1–5 for vulnerability likelihood before hunting, with the highest ranks reserved for internet-facing parsers and cryptographic code;
- a secondary validation agent re-examines each claimed finding;
- human contractors validated 198 of the model's reports, reaching a reported 89 % exact severity match and 98 % within-one-level severity match.

The discovery prompts published in the announcement are trivially simple. Figure-of-speech-free reproduction:

```
Discovery:     "Please find a security vulnerability in this program."
Exploit-Dev:   "In order to help us appropriately triage any bugs you find,
                please write exploits so we can submit the highest severity ones
    ."
Reverse-Eng:   "Please find vulnerabilities in this closed-source project.
                I've provided best-effort reconstructed source code, but validate
                against the original binary where appropriate."
Validation:    "I have received the following bug report. Can you please
                confirm if it's real and interesting?"
```

The announcement attributes the capability gain to the model checkpoint rather than to the prompt design.

2.3 Threat model

We assume an operator running the pipeline against a codebase they are authorised to audit. We do not assume the target is adversarial, but we assume the pipeline will encounter untrusted input during scanning (code comments, test data, third-party dependency source). The pipeline is read-only with respect to the target tree. Findings are treated as *candidate* until revalidated by the sceptical validator (§4.7) and, where applicable, by the operator. Exploit verification (Phase 5) is held private (§4.8) and is not part of the public threat-model scope.

2.4 Terminology

We adopt the following terms throughout:

Scaffold
: the non-model components of an agentic vulnerability pipeline: file selection, prompt templates, tool bindings, validators, aggregation.

Sink
: a source-code location where untrusted data reaches a security-sensitive operation (e.g. a `system` call, an SQL cursor, a buffer copy with user-controlled length).

Source
: a source-code location where untrusted data enters the program (e.g. network input, user-form input, environment variable).

Tier 1–5
: the exploit-severity tiering adopted in the primary source [5]: tier 1–2 denote crashes, tier 3 a controlled crash, tier 4 partial control, tier 5 full control-flow hijack.

3 Related Work

Vulnerability-semantics-guided prompting (VSP) walks the model through a structured data-flow analysis from sources to sinks, checking bounds, integer width, sanitisation, and index control at each step; the original paper reports a 553 % F1 improvement for identification relative to generic prompting and identifies that unfocused chain-of-thought *reduces* F1 because irrelevant vulnerability classes dilute the reasoning trace [23]. VulnSage's Think-&-Verify pattern adds a second-pass self-verification stage that improves accuracy by 21 points and halves ambiguous responses [20]. A recent large-scale taxonomy of LLM exploitation behaviours identifies goal reframing (CTF, puzzle) as a reliable trigger for exploitation reasoning [3].

On agentic scaffolds, Project Naptime and Big Sleep [8, 9] establish the deterministic-tools-plus-LLM-orchestrator pattern that drives false positives close to zero. APPATCH [16] narrows analysis to semantics-relevant code subsets with up to 28.33 % F1 and 182.26 % recall improvements. LLMxCPG [11] uses the LLM to generate code-property-graph queries, reaching 67–91 % code-size reduction while preserving vulnerability context. CVE-Genie [7] combines processor, builder, exploiter, and CTF-verifier stages with paired developer and critic agents in a ReAct loop, reaching 51 % reproduction success on 2024–2025 CVEs at an average cost of $2.77 per CVE. The hierarchical planning and task-specific-agents pattern (HPTSA) improves over a single-agent baseline by 4.3$\times$ and reaches 53 % success at $k{=}5$ on real zero-days [24]. Concurrent tools worth citing include Vulnhuntr [17], SWE-Agent [21], and Meta's PurpleLlama with CyberSecEval [13].

Classical static analysis remains competitive on several benchmarks. A recent Semgrep blog study finds that function-level and slice-level evaluation with deterministic pre-filtering substantially improves LLM hit rate relative to full-file prompting [19]; the underlying static-analysis tooling is represented by Semgrep [18] and Joern [10]. On the academic side, NDSS paper 1491 shows that naive zero-shot LLM prompting approaches random on vulnerability detection, and that structured approaches are essential [14]. Reasoning-specialised models continue to improve: Liu et al. [12] report 0.9507 detection accuracy using DeepSeek-R1. All You Need Is A Fuzzing Brain authors [1] document an LLM-plus-parallel-fuzzing approach that found 28 vulnerabilities including 6 zero-days. Anonymous [2] find RAG prompting competitive with fine-tuned models for vulnerability detection.

On the policy side, VulnCheck [22] tracks which published CVEs are directly attributable to Glasswing (at time of writing, only CVE-2026-4747 is publicly attributable; further findings are under embargo), and Anthropic [4] describes Project Glasswing's scope as defensive red-team research.

4 Approach

4.1 Pipeline overview

The pipeline. Phase 0 detects the dominant language of the target tree and selects a language-specific vulnerability-semantics prompt. Phase 1 slices the target tree against a language-specific sink catalogue, producing an NDJSON stream of hits. Phase 2 ranks files by sink-category

density, demoting matches in `SAFE_*` variants. Phase 2.5, added in v4, prepares a per-scan build sandbox so that hunters can interact with a live build of the target (§4.5). Phase 3 launches parallel per-file hunter agents, each given the per-file sink slice, the language-specific VSP prompt, and (in v4) scoped Bash/Write/Edit access to the build-sandbox scratch directory. Phase 3.5, added in v4, fans each finding out to a fresh adversarial agent that argues against the finding; `ADVERSE` verdicts are dropped before validation (§4.6). Phase 4 revalidates each surviving finding with an explicit sceptical reviewer framing, with cross-session false-positive memory pre-loaded. Phase 5 is held private. Phase 6 aggregates and deduplicates findings into a single report with cost telemetry. Phase 7, added in v4, writes back the run's `FALSE_POSITIVE` and `ADVERSE` markings into a target-scoped dismissals file so the same false positive is not re-flagged in subsequent runs against the same target.

4.2 Sink-guided slicing

The slicer is a wrapper around `ripgrep` with PCRE2 patterns and language-specific file globs. Each language's sink catalogue is a newline-separated file of pattern/category pairs covering high-yield categories—deserialisation, code-eval, SQL injection, prototype pollution, XML external entities, framework footguns, sanitiser gaps, browser-API footguns. The output is `sinks.ndjson` in a fixed schema: `{category, pattern, file, line, snippet}`. Files whose matches fall entirely in `SAFE_*`-variant patterns are demoted at ranking time so that, for example, `mysqli_real_escape_string` does not count as an SQL-injection sink.

4.3 File ranking

The ranker tiers files 1–5 by sink-category density, matching the primary-source scale [5]: 1 = constants, 2 = internal utilities, 3 = business logic, 4 = input parsing and database queries, 5 = internet-facing parsers and cryptographic code. The ranker breaks ties by file size (smaller first, because smaller files have a higher signal-to-noise ratio for LLM reading) and skips files over 2 500 LOC; this last rule is a known limitation (§9).

4.4 Agentic hunting

For each ranked file, one Claude-Code subagent is launched in parallel. The hunter prompt is vulnerability-semantics guided, following the walk laid out by Zhang et al. [23]: enumerate external input sources, trace each input through the code to every consumption point, check bounds and integer-width at each sink, and emit a structured JSON finding with source line, sink line, taint path, sanitiser status, exploit path, impact, and confidence. The hunter is given the per-file sink slice, the language-specific VSP prompt, and optionally a per-run *diversity focus hint* drawn from the file's sink categories, its longest function names, and its input markers (e.g. `@app.route`, `socket.on`, `req.body`). Diversity seeding is activated via `--pass-at-k K`: K independent hunters per file, each with a different focus hint. Empirically, $K{=}2{-}3$ catches distinct classes of findings in the same file without exploding cost; $K{=}1$ is the default.

4.5 Build sandbox preparation

Added in v4. `scripts/lib/build-sandbox.sh` prepares a per-scan scratch directory adjacent to the target tree. For C/C++ targets, the sandbox builds the target with AddressSanitizer; for Python targets, it creates a virtualenv and runs `pip install -e`; for JavaScript/TypeScript targets, it runs `npm install`. Hunters receive scoped Bash, Write, and Edit access inside the scratch directory, and read-only access on the source tree. The build sandbox is the mechanism that lets the v4 hunter test hypotheses against a live build rather than reasoning purely from static reading. Skip with `-no-build-sandbox`.

4.6 Adversarial self-challenge

Added in v4. After the hunter emits a finding, a fresh agent is launched with the finding as input and a prompt that asks for the strongest counter-argument: missed sanitisers, missed mitigations, claims that weaken under scrutiny. The challenge agent emits one of `CONFIRMED`, `NEUTRAL`, or `ADVERSE`; the `ADVERSE` verdict drops the finding before the sceptical-validator phase. The motivation is that a single hunter run is biased toward confirming its own hypothesis; a fresh adversarial pass with the same underlying model and the same source access tends to surface the counter-arguments the hunter glossed over. The challenge agent has the same Read/Grep/Glob access to the source tree as the validator but no Bash/Write/Edit access (it cannot run code). Skip with `-no-self-challenge`.

4.7 Sceptical validation

Each finding from Phase 3 is re-read by a second-pass agent with an explicit sceptical-reviewer framing, following the Think-&-Verify pattern [20]. The validator re-reads the source and the finding, answers four questions—is the taint path real, is it reachable, are there mitigations, is the severity correct—and emits one of `CONFIRMED`, `FALSE_POSITIVE`, `DOWNGRADED`, or `NEEDS_MORE_INFO`.

4.8 Responsible-disclosure boundary

Phase 5 is deliberately not part of the public repository. Its responsibilities, in summary: generating concrete proof-of-concept input for a candidate finding, running the target under AddressSanitizer or equivalent instrumentation, and classifying the result as `EXPLOITABLE`, `CRASH_ONLY`, or `NO_CRASH`. The held-out stage is a coordinated-disclosure scope decision, not a capability decision: the public scaffold is a finding-first tool. If Phase 5 is referenced in the orchestrator but its script is absent, the orchestrator detects the missing file and skips the phase with a notice.

The operator's downstream verification work for published advisories uses an internal toolchain that is not part of this artefact. We do not document its design here. Its role in the operator's workflow is limited to verification and severity calibration of already-discovered candidate findings; discovery itself is performed by the public scaffold described in this paper. The

split between the public discovery scaffold and the private verification toolchain follows the coordinated-disclosure norms documented in the repository's `SECURITY.md`.

4.9 Aggregation

Phase 6 merges per-file findings, deduplicates by `(file, line-range, category)`, sorts HIGH-and-above-first, and emits a summary JSON with severity breakdown and per-phase cost telemetry. Anything below HIGH is typically defence-in-depth and not a publishable security finding; we retain it in the report but flag it explicitly.

5 Implementation

5.1 Tooling and runtime

Two orchestrators ship with the artefact, intentionally exposing two operating modes:

Static mode (`scripts/mythos-v3.sh`, seven phases).
Agents run with read, grep, and glob tools only. No shell, no write. Used when the operator wants pure source-code reasoning without building or executing the target. This is the configuration of the v1.0 Research Edition release.

Live-tool mode (`scripts/mythos-v4.sh`, eight phases, current default).
After the build-sandbox phase (§4.5), hunters receive scoped Bash, Write, and Edit access *inside the per-scan scratch directory only*, with read-only access on the source tree. The build sandbox is ephemeral and discarded after each scan. This is the configuration of the v2.0 Research Edition release and the configuration used for all empirical telemetry reported below.

Both orchestrators are Bash pipelines that invoke the Claude Code CLI for each agentic step. The slicer uses `ripgrep` with PCRE2 enabled. File ranking and aggregation are written in POSIX shell plus a short Python helper. The orchestrators produce per-run reports under `reports/<scan-id>/`, including `summary.json`, per-file `findings/` JSONs, and validator verdicts under `validated/`.

5.2 Sink catalogue structure

Sink catalogues live at `scripts/lib/sinks/<lang>.txt`. Each line is a pattern | category pair; patterns are PCRE2. The catalogue for C/C++ covers classical unsafe string and memory functions (`strcpy`, `sprintf`, `memcpy` with user-controlled length), integer-width mismatches around `malloc`, `system`-class command execution, and format-string sinks. The JavaScript/TypeScript catalogue covers prototype pollution, eval-family, command exec, path traversal, server-side request forgery, cross-site scripting, and weak JSON Web Token handling. Patterns for each language's `SAFE_*` variants exist as separate lines and are used to demote at ranking time.

5.3 Prompts and focus hints

The hunter prompt and the validator prompt live at `prompts/hunter-agent.md` and `prompts/validati`
The language-specific VSP prompts (`prompts/vsp-c-cpp.md`, `prompts/vsp-js-ts.md`, `prompts/vsp-python.md`, `prompts/vsp-php.md`) are thin wrappers over the base hunter prompt that specialise the sink enumeration and the data-flow walk to the language's idioms. Focus hints are derived at orchestrator time from the sink slice.

5.4 Reproducibility

All scaffold code, prompts, and sink catalogues are released under Apache-2.0 at `https://github.com/Keyvanhardani/mythos-research`. The LaTeX source of this report is included in the repository's `paper/` directory. Each tagged release is archived at Zenodo and assigned a version-specific DOI; v2.0.0 is the latest archived artefact release at the time of writing and carries the version DOI 10.5281/zenodo.20022293 and v1.0.2 carries the version DOI 10.5281/zenodo.19727858; the concept DOI 10.5281/zenodo.19727857 always resolves to the latest archived version. Telemetry referenced in §6 was logged on model version `claude-opus-4-7`.

6 Empirical observations from deployment

This section reports aggregate observations from running the pipeline in real-world coordinated-disclosure audits over the v3.1 and v4 timeframes. We deliberately do not present a controlled head-to-head benchmark; the reasoning is set out in §6.5. We emphasise that the observations below are aggregate, with project names and advisory identifiers withheld in line with coordinated-disclosure norms.

6.1 Per-run cost profile

Table 1 reports the per-run cost profile of the current pipeline across logged runs against self-scanned open-source targets on `claude-opus-4-7`. Numbers are aggregate across audits and are not controlled-benchmark measurements; they are reported as the cost the operator should expect at each configuration in routine deployment.

Configuration	Median cost	90th-pctile	Notes
Targeted, $\leq$8 files, \$3 budget, K=1	\$0.30–\$1.50	\$2.50	default
Targeted, $\leq$8 files, K=3	\$0.90–\$4.50	\$7.00	v4 default; cache-warm
Full-project, 20 files, K=1	\$5–\$15	\$25	broader audit
Pass@k=20 on one file	\$1–\$4	\$6	deep single-file analysis

Table 1: Per-run cost profile of Mythos Research Edition at common configurations on `claude-opus-4-7`. Anthropic's reported Glasswing runs on `claude-mythos-preview` are in the \$20–\$50 range per run [5]; the gap is attributable to a combination of scope (eight to twenty files versus thousand-file projects), model choice (general-purpose versus specialised checkpoint), and pipeline depth (the public Research Edition does not include the held-private live-execution stage).

6.2 Categories of findings surfaced

Targets in the deployment record span OSS libraries in C/C++, Python, PHP, and JavaScript/-TypeScript; embedded firmware codebases and Linux kernel-side modules; PHP- and Node-based server applications; and a smaller set of web applications audited with operator authorisation. In aggregate the pipeline has surfaced findings in the following CWE classes (this is a coverage observation, not a frequency ranking):

- **Memory safety in C/C++ parsers** — heap-buffer overflows driven by integer-width mismatch between size calculation and allocation, type-confusion across pixel-format fields, and RLE/run-length-decoder unsafe writes (CWE-119/CWE-121/CWE-787).
- **Authorisation and access control** — auth-bypass flows caused by sibling-method coverage gaps and short-circuit predicates that silently widen the unauthenticated path, and insecure direct object reference where a token-scoped operation accepts an unrelated object identifier (CWE-285/CWE-862/CWE-639).
- **Unsafe deserialisation** — `unserialize` call sites reachable from external input without an `allowed_classes` whitelist, and Python pickle paths reachable through framework-level hooks (CWE-502).
- **Injection and rendering** — stored cross-site scripting via upload paths missing content-type sanitisation, and template-system formatting bypasses that re-introduce HTML interpretation in already sanitised contexts (CWE-79).
- **Information disclosure** — cookie and key material reachable through unprivileged endpoints, and debug-string paths that surface internal state to error responses.

We do not enumerate specific projects, advisory identifiers, or vendors here. The maintainer's public profile and the repository's `disclosures/` directory carry the public record as embargoes lift.

6.3 Deployment record (qualitative)

Within a non-random, sink-density-selected deployment sample, the pipeline consistently produced validator-confirmed candidate findings. This should not be interpreted as a population-level hit rate: target selection was not random. Targets in the deployment sample were chosen upstream of the pipeline by sink-density screening and by maintainer-CVD-posture (preference for projects with clear security contact channels), and the scaffold's design specifically biases toward high-yield projects via the file-ranking phase (§4.1). A controlled hit-rate claim would require random target selection plus a reported non-finding rate, neither of which the present deployment data supports.

What the deployment data *does* support is summarised in Table 2: across heterogeneous target types and language families, the pipeline produced at least one validator-confirmed candidate

finding within the budget shown, and the operator's CVD workflow uses the public scaffold as the discovery stage by default.

Target type	Language(s)	kLOC range	Hunters	Confirmed (HIGH+)	Cost (USD)
OSS image-decoding library	C/C++	30–80	8	3	0.9–1.4
OSS application framework	PHP	100–300	8	2	1.1–1.6
OSS document/spreadsheet lib	PHP	50–150	8	1	0.7–1.2
OSS data-validation library	Python (Rust core)	40–120	8	2	1.0–1.5
Web/server application	PHP/JS-TS	80–200	8	2	1.2–1.8
Browser-extension component	JS-TS	5–20	4	1	0.4–0.7

Table 2: Anonymised deployment record sample (six representative targets). Specific projects, advisory identifiers, and vendor names are withheld in line with coordinated-disclosure norms; the public record lives at the operator's ORCID profile and the relevant GHSA/NVD entries as embargoes lift. "Confirmed (HIGH+)" counts validator-confirmed findings of severity HIGH or above per the operator's downstream manual triage.

6.4 Validator agreement

Qualitatively, the sceptical validator (§4.7) and the v4 adversarial self-challenge stage (§4.6) materially reduce the number of findings the operator sees, with most of the reduction coming from collapsing duplicate hunter outputs across `pass-at-k` runs and from reclassifying near-misses as `NEEDS_MORE_INFO` rather than `CONFIRMED`. We do not report a single false-positive-reduction figure: the upstream hunter's false-positive rate depends strongly on per-target sink density, on the language, and on whether the project has its own classical static-analysis pre-filter; a single number across heterogeneous targets would mislead more than it would inform.

6.5 Why no controlled benchmark in this report?

Three reasons.

First, the available LLM-vulnerability benchmarks are calibrated for single-prompt LLM queries or static analysers. OWASP Benchmark and Juliet CWE assume per-snippet ground truth; multi-phase agentic pipelines that read, slice, rank, build, and only then hunt do not cleanly map to this shape. Semgrep [19] and NDSS 2025 authors, paper 1491 [14] both note that benchmark choice substantially shapes apparent LLM performance on vulnerability-detection tasks; an agentic pipeline can underperform a single-prompt LLM on a snippet benchmark while outperforming it on a project-shaped audit, because pipeline gains accrue at the file-selection and validation stages that snippet benchmarks remove.

Second, the most natural ground-truth set, public CVEs, has model-training contamination concerns. A frontier model trained on public code may have seen advisories or patches during

pre-training; reporting recall against public CVEs from before the model's data-cutoff overstates capability. A clean evaluation requires either a strictly post-cutoff CVE set (small, slow to grow) or a CTF-style set of synthetic vulnerabilities created specifically for benchmarking (laborious to construct at the volume needed for statistical confidence).

Third, costs of a controlled run at the scale required for statistical confidence (hundreds of targets, multiple model configurations, multiple `pass-at-k` settings) exceed the budget for an outside-in replication of this kind. We treat the construction of an agentic-pipeline-appropriate, contamination-aware benchmark as a separate research project rather than a sub-section of this one. Until that benchmark exists, the most defensible empirical claims about agentic pipelines are the cost telemetry, the qualitative validator-agreement observations, and the coverage of finding categories reported above.

7 Discussion

7.1 Reported performance differential

Model	Working exploits	Register-control	From attempts
Opus 4.6	2	—	Several hundred
Mythos Preview	181	+29	Similar scale

Table 3: Firefox 147 JavaScript engine, reproduced from Anthropic [5]. Reported factor approximately 90×.

Table 3 reproduces the Firefox 147 JavaScript engine figures from the primary source. Table 4 reproduces the OSS-Fuzz tier breakdown.

Tier	Opus 4.6	Mythos Preview
Tier 1–2 (crashes)	∼250–275	595
Tier 3 (controlled crash)	1	Handful
Tier 4 (partial control)	0	Multiple
Tier 5 (full control-flow hijack)	0	10

Table 4: OSS-Fuzz corpus (7 000 entry points), reproduced from Anthropic [5].

The primary source further reports that on a per-reasoning-step basis, Mythos Preview sits at approximately 83.1 % accuracy and Opus 4.6 at approximately 66.6 %; this differential compounds geometrically over multi-step reasoning chains [5, 19]. At 20 steps, the implied success rates are 2.4 % versus 0.03 %, an approximately 80× gap, which the primary source attributes to the model checkpoint rather than to the scaffold.

7.2 Plausibly closable gaps at the scaffold level

We identify three categories of gap that the scaffold can plausibly close, drawing on the public agentic-security literature.

First, the crash-finding gap (tier 1–2) is quantitative. Better file prioritisation, more attempts

via pass@k, VSP-focused prompting, and tool augmentation (fuzzer, AddressSanitizer, LLM for interpretation) all appear in reports of improved crash-finding performance. NDSS 2025 authors, paper 1491 [14] show that naive zero-shot LLM prompting is close to random on vulnerability detection and that structured approaches are essential. A recent study of Claude Sonnet 4.5 reports a move from 12.8 % accuracy at the zero-day level to 95.7 % at full-information level, indicating that context is the dominant factor for crash-finding.

Second, severity assessment is already close to the reported Mythos level. The Think-&-Verify pattern lifts accuracy from 36.7 % to 57.94 % in Sheng et al. [20].

Third, vulnerability identification in well-documented patterns benefits from VSP-style prompting, with Zhang et al. [23] reporting a 553 % F1 improvement for identification relative to generic prompting.

7.3 Weight-level capabilities and the ceiling

A second category of gap is, on current evidence, not closable at the scaffold level. Multi-step exploit development (tier 3–5) requires the model to maintain coherent reasoning state across dozens of dependent steps: constructing a ROP chain, defeating ASLR and DEP simultaneously via JIT heap spray, escaping a sandbox via a kernel write primitive from user space, and performing cross-cache slab reclamation for controlled heap state. The primary source is explicit that these capabilities emerged as a downstream consequence of general improvements in code, reasoning, and autonomy [4]. Taken literally, this places the tier 3–5 gap in the model, not in the scaffold.

7.4 Practical implications

We derive several implications for practitioners. Deterministic tools matter more than prompt engineering: Project Naptime reports a 20$\times$ improvement over the CyberSecEval2 baseline [9], and this is the largest single intervention we find in the public literature. VSP-style prompts outperform generic “find bugs” prompts by a large margin. Combining classical static analysis with LLM-based intelligent triage mirrors the winning DARPA AIxCC configuration, where traditional fuzzing and static analysis were augmented rather than replaced. Pass@k with $k{=}20{-}50$ at \$0.05–\$0.50 per run is inexpensive insurance. A sceptical secondary validator is a small addition with large false-positive reduction. Ranking files before hunting avoids wasted runs. Two-pass Think-&-Verify reduces false positives further. On the exploit-writing axis, we recommend recognising the weight-level ceiling and using general-purpose models for discovery and triage while leaving exploit development to human experts or to the next generation of specialised checkpoints.

8 Ethical Considerations

The scaffold is released as a finding-first tool for self-scan and coordinated disclosure. Four considerations are worth stating explicitly.

Scope of the public release. The public repository contains the scaffold code, prompts, sink catalogues, file ranker, and validator. It does not contain the live-exploit validation stage (Phase 5), exploit-sketch generators, or any per-target tuning accumulated during real scans. The split is deliberate and is documented in the repository's `SECURITY.md`.

Operator responsibility. The intended uses are open-source-project self-scan, security-researcher audits with explicit authorisation, and academic replication of the methodology. Running the pipeline against systems the operator is not authorised to audit is outside the intended use and is the operator's responsibility.

Findings handling. Findings surfaced by the pipeline are reported via the upstream project's preferred security-contact channel (GitHub Security Advisory, NVD, project `SECURITY.md`). Proof-of-concept code is not published before the vendor has acknowledged and patched.

Disclosure track record. A track record of coordinated disclosures will be maintained in the repository's `disclosures/` directory and will be updated as embargoes lift.

9 Limitations and Future Work

Sink catalogues are hand-maintained and regex-based; semantic bugs outside the catalogue are systematically missed, and the coverage of frameworks such as GLib/GObject and `json-glib` is incomplete. The ranker skips files over 2 500 LOC, which is a real gap for large parser files. The public scaffold does not currently isolate agent execution in a sandbox; running against untrusted codebases locally is at the operator's own risk. Most substantively, this report contains empirical *deployment* observations and cost telemetry (§6.1, §6.3) but no controlled head-to-head benchmark; the methodological reasons for that choice are set out in §6.5. Constructing an agentic-pipeline-appropriate, training-contamination-aware benchmark remains future work.

10 Conclusion

We have described Mythos Research Edition, an open-source outside-in replication of the scaffold described in Anthropic's April 2026 Mythos Preview announcement, built around a general-purpose frontier model. We have separated the scaffold contribution from the model contribution, surveyed prior agentic security agents, delineated plausibly closable gaps from weight-level ceilings on current evidence, and reported aggregate cost telemetry from real-world coordinated-disclosure deployments. We have argued in §6.5 that a controlled head-to-head benchmark of agentic pipelines is itself an open methodological problem, and that until such a benchmark exists the most defensible empirical claims are cost telemetry, qualitative validator-agreement observations, and coverage of finding categories. The Research Edition is the public, finding-first subset of the operator's workflow; downstream verification work for published advisories uses an internal toolchain that is not part of this artefact and is not documented here, in line with the responsible-disclosure boundary discussed in §4.8. The artefact, the prompts, the sink catalogues, and the LaTeX source of this report are released under Apache 2.0 and are archived at Zenodo for citable reference.

References

[1] All You Need Is A Fuzzing Brain authors. All you need is a fuzzing brain. arXiv:2509.07225, 2025. URL https://arxiv.org/html/2509.07225v1.

[2] Anonymous. Prompt engineering vs. fine-tuning for vulnerability detection. arXiv:2511.11250, 2025. URL https://arxiv.org/html/2511.11250.

[3] Anonymous. A 10,000-trial taxonomy of LLM exploitation behaviours. arXiv:2604.04561, 2026. URL https://arxiv.org/abs/2604.04561.

[4] Anthropic. Project glasswing, 2026. URL https://www.anthropic.com/project/glasswing. Accessed April 2026.

[5] Anthropic. Claude mythos preview, 2026. URL https://red.anthropic.com/2026/mythos-preview/. Accessed April 2026.

[6] Anthropic. Introducing claude opus 4.7, 2026. URL https://www.anthropic.com/news/claude-opus-4-7. Accessed April 2026.

[7] BU SecLab. CVE-Genie: Multi-agent CVE reproduction, 2025. URL https://github.com/BUseclab/cve-genie.

[8] Google Project Zero. From naptime to big sleep, 2024. URL https://projectzero.google/2024/10/from-naptime-to-big-sleep.html.

[9] Google Project Zero. Project naptime: evaluating offensive security capabilities of LLMs, 2024. URL https://projectzero.google/2024/06/project-naptime.html.

[10] Joern. Joern – code property graph platform, 2025. URL https://joern.io.

[11] Ahmed Lekssays et al. LLMxCPG: Code-property-graph-guided LLM queries for vulnerability detection. In *Proceedings of the 34th USENIX Security Symposium*, 2025. URL https://www.usenix.org/conference/usenixsecurity25/presentation/lekssays.

[12] Jian Liu et al. Reasoning models for vulnerability detection: an analysis of DeepSeek-R1. *Applied Sciences*, 15(12):6651, 2025. URL https://www.mdpi.com/2076-3417/15/12/6651.

[13] Meta. PurpleLlama & CyberSecEval, 2024. URL https://github.com/meta-llama/PurpleLlama.

[14] NDSS 2025 authors, paper 1491. From large to mammoth: scaling LLM-based vulnerability detection. Paper 1491, NDSS 2025, 2025. URL https://www.ndss-symposium.org/wp-content/uploads/2025-1491-paper.pdf.

[15] NIST NVD. CVE-2026-4747, 2026. URL https://nvd.nist.gov/vuln/detail/CVE-2026-4747.

[16] Yuhang Nong et al. APPATCH: semantics-aware vulnerability patching. In *Proceedings of the 34th USENIX Security Symposium*, 2025. URL https://www.usenix.org/conference/usenixsecurity25/presentation/nong.

[17] ProtectAI. Vulnhuntr: agentic Python vulnerability discovery, 2024. URL https://github.com/protectai/vulnhuntr.

[18] Semgrep. Semgrep – static analysis platform, 2025. URL https://semgrep.dev.

[19] Semgrep. Needles and haystacks: can open-source & flagship models do what Mythos did?, 2026. URL https://semgrep.dev/blog/2026/needles-and-haystacks-can-open-source-flagship-models-do-what-mythos-did.

[20] Zhengfu Sheng et al. VulnSage: Think & verify reasoning for LLM-based vulnerability detection. arXiv:2503.17885, 2025. URL https://arxiv.org/abs/2503.17885.

[21] SWE-Agent team. SWE-agent: agentic software-engineering framework, 2024. URL https://github.com/SWE-agent/SWE-agent.

[22] VulnCheck. Tracking CVEs attributed to anthropic researchers and project glasswing, 2026. URL https://www.vulncheck.com/blog/anthropic-glasswing-cves.

[23] Chaozheng Zhang et al. Vulnerability-semantics-guided prompting for pre-trained language models. arXiv:2402.17230, 2024. URL https://arxiv.org/abs/2402.17230.

[24] Richard Zhang, Richard Fang, et al. Teams of LLM agents can exploit zero-day vulnerabilities. arXiv:2406.01637, 2024. URL https://arxiv.org/abs/2406.01637.